

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: Dwarapudi Suswetha

Roll No.: A24126552079

Date: 20-09-2026

1. TITLE

Develop an Interactive Profile Customizer Using JavaScript DOM Manipulation

2. AIM

To develop an interactive Profile Customizer using HTML, CSS and JavaScript, and demonstrate how JavaScript can dynamically change the content, style and attributes of HTML elements using the Document Object Model (DOM).

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of the Document Object Model (DOM).
 - To access HTML elements using JavaScript.
 - To dynamically change the content of HTML elements.
 - To dynamically modify the CSS styles of elements.
 - To modify HTML attributes using JavaScript.
 - To handle button click events using JavaScript functions.
 - To dynamically change the page theme using CSS classes.
 - To understand how JavaScript interacts with the structure of a web page. •
To create an interactive profile customization interface.
-

4. THEORY

The **Document Object Model (DOM)** is a programming interface that represents an HTML document as a tree of objects. It allows JavaScript to access, modify and update HTML elements dynamically.

Using the DOM, JavaScript can change:

- The content of an HTML element.
- The CSS style of an HTML element.
- The attributes of an HTML element.
- The classes applied to an HTML element.
- The overall appearance of a web page.

In this application, JavaScript uses the `document.getElementById()` method to access elements such as the profile name, role, description, avatar and status message.

The `innerText` property is used to change the text content of an element dynamically.

The `style` property is used to directly modify CSS properties such as:

- `color`
- `fontSize`
- `background`
- `borderColor`
- `transform`

The `setAttribute()` method is used to modify an HTML attribute. In this application, it changes the title attribute of the avatar.

The `classList.toggle()` method is used to add or remove the dark class from the `<body>` element. This allows the application to switch between light and dark themes.

The basic flow of the application is:

User Click → JavaScript Function → DOM Element Access → Content/Style/Attribute Modification → Updated Web Page

5. ALGORITHM / PROCEDURE

1. Create a new HTML document.
2. Define the basic HTML structure using `html`, `head` and `body` elements.
3. Create a profile card containing an avatar, name, role and description.
4. Add buttons for changing the name, role, bio, avatar and theme.
5. Use CSS to style the profile card and create a pixel-style avatar.
6. Define a dark theme using the `.dark` CSS class.
7. Create JavaScript functions for modifying the profile.
8. Use `document.getElementById()` to access the required HTML elements.
9. Create the `changeName()` function to modify the profile name.
10. Create the `changeRole()` function to modify the role and its color.
11. Create the `changeDescription()` function to modify the bio and font size.

12. Create the changeAvatar() function to modify the avatar background, border, transformation and title attribute.
 13. Create the changeTheme() function to toggle the dark theme.
 14. Use innerText to dynamically change the content of elements.
 15. Use the style property to dynamically modify CSS properties.
 16. Use setAttribute() to modify the avatar's title attribute.
 17. Use classList.toggle() to switch the page theme.
 18. Display a status message after each DOM operation.
 19. Open the HTML file in a web browser.
 20. Click each button and verify the changes.
 21. Compare the expected and actual results.
-

6. PROGRAM

Code.html

```
<!DOCTYPE html>
<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport"
content="width=device-width, initial-scale=1.0">

  <title>DOM Profile Customizer</title>
```

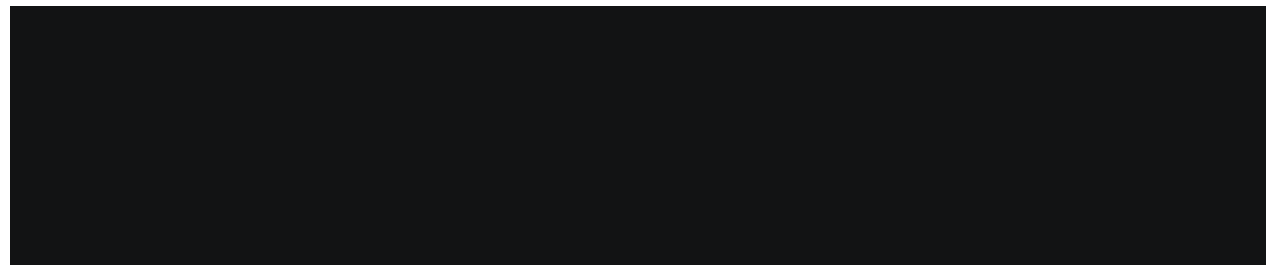
```
<style>
```

```
* {
```

```
    box-sizing: border-box;  
}
```



```
body { margin: 0;
min-height: 100vh; font-
family: Arial, sans-serif;
background: #eee8df;
```



```
        display: flex;        justify-
content: center;        align-
items: center;        transition:
0.4s;
    }

    .card {        width: 420px;        background:
#ffdfd9;        padding: 30px;        border-radius:
18px;        text-align: center;        box-shadow: 0
10px 30px rgba(60, 45, 30, 0.15);        transition:
0.3s;
    }

    .card:hover {
transform: translateY(-5px);
    }

    /* Pixel Avatar */

    .avatar {        width:
120px;        height:
120px;        margin:
auto;        border-radius:
18px;
```



background:
linear-gradient(#3c3029 0 0)
30px 15px / 60px 15px no-repeat,

linear-gradient(#e8b28b 0 0)
30px 30px / 60px 60px no-repeat,

linear-gradient(#3c3029 0 0)
30px 90px / 60px 15px no-repeat,

linear-gradient(#e8b28b 0 0)



```
15px 45px / 15px 30px no-repeat,
```

```
linear-gradient(#e8b28b 0 0)  
90px 45px / 15px 30px no-repeat,
```

```
#f3d7c1;  
position: relative;  
border: 5px solid #d17a45;
```

```
box-shadow:  
0 5px 15px rgba(70, 50, 35, 0.15);  
}
```

```
.avatar::before { content:  
""; position: absolute;  
width: 10px; height: 10px;  
background: #3d3027; left:  
42px; top: 53px; box-  
shadow: 25px 0 #3d3027;  
}
```

```
.avatar::after {  
content: "";
```



```
        position: absolute;
width: 20px;      height:
5px;      background:
#9f5a45;      left: 45px;
top: 75px;
    }      h1 {
margin: 18px 0 5px;
color: #3d3027;
    }
```



```
    #role {
color: #a4613a;
font-weight: bold;
    }

    #description {
color: #75675d;
line-height: 1.6;
    }

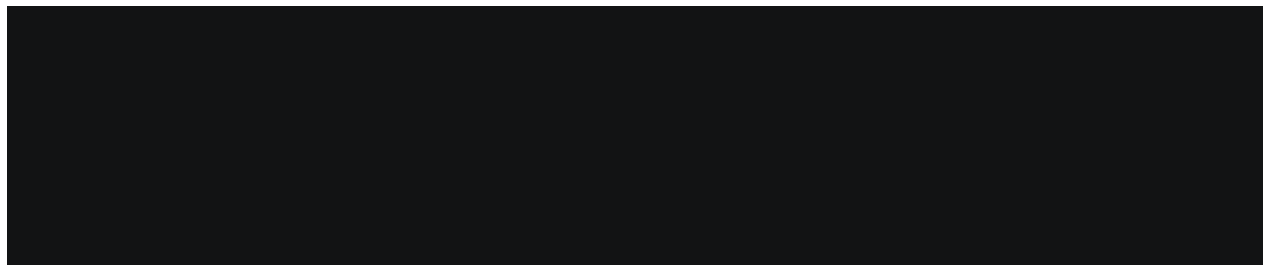
    .buttons {
margin-top: 25px;
display: flex;        flex-
wrap: wrap;          justify-
content: center;      gap:
8px;
    }
    button {          border:
none;                padding: 10px
14px;                border-radius:
8px;                background:
#c56a35;            color: white;
cursor: pointer;
transition: 0.2s;
```



```
}
```

```
    button:hover {  
background: #9f5028;  
transform: scale(1.04);  
    }
```

```
    .status {        margin-  
top: 20px;          padding:  
10px;              border-radius:  
8px;              background:  
#f4eee7;
```



```
        color: #66574c;
font-size: 14px;
    }

    /* Dark theme */

    .dark {
background: #292622;
    }

    .dark .card {
background: #3a3530;
    }

    .dark h1 {
color: #fffaf4;
    }

    .dark #description {
color: #d8cec4;
    }

    .dark .status {
background: #484139;
color: #eee3d8;
```

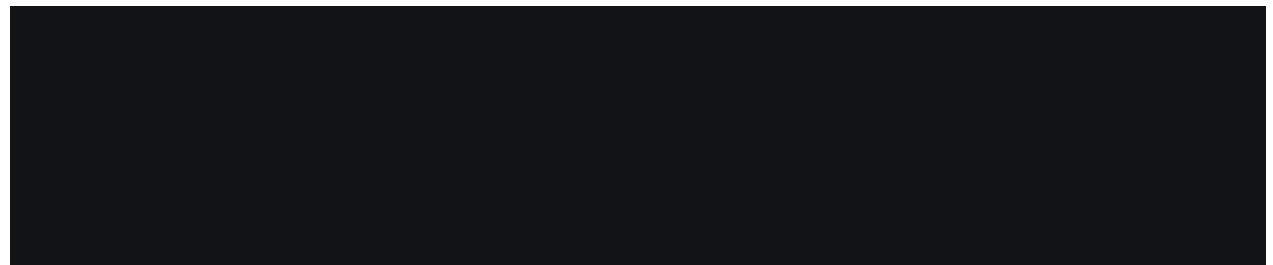
[REDACTED]

}

[REDACTED]



```
.dark #role {  
color: #e39a6d;  
}  
  
</style>  
  
</head>  
  
<body>  
  
  <div class="card">
```



```
<!-- Faceless Pixel Avatar -->

<div
class="avatar"
id="avatar"
title="Pixel avatar">
  </div>

<h1 id="name">
  Mohammed Almas
</h1>

<div id="role">
  AI & Full Stack Developer
</div>

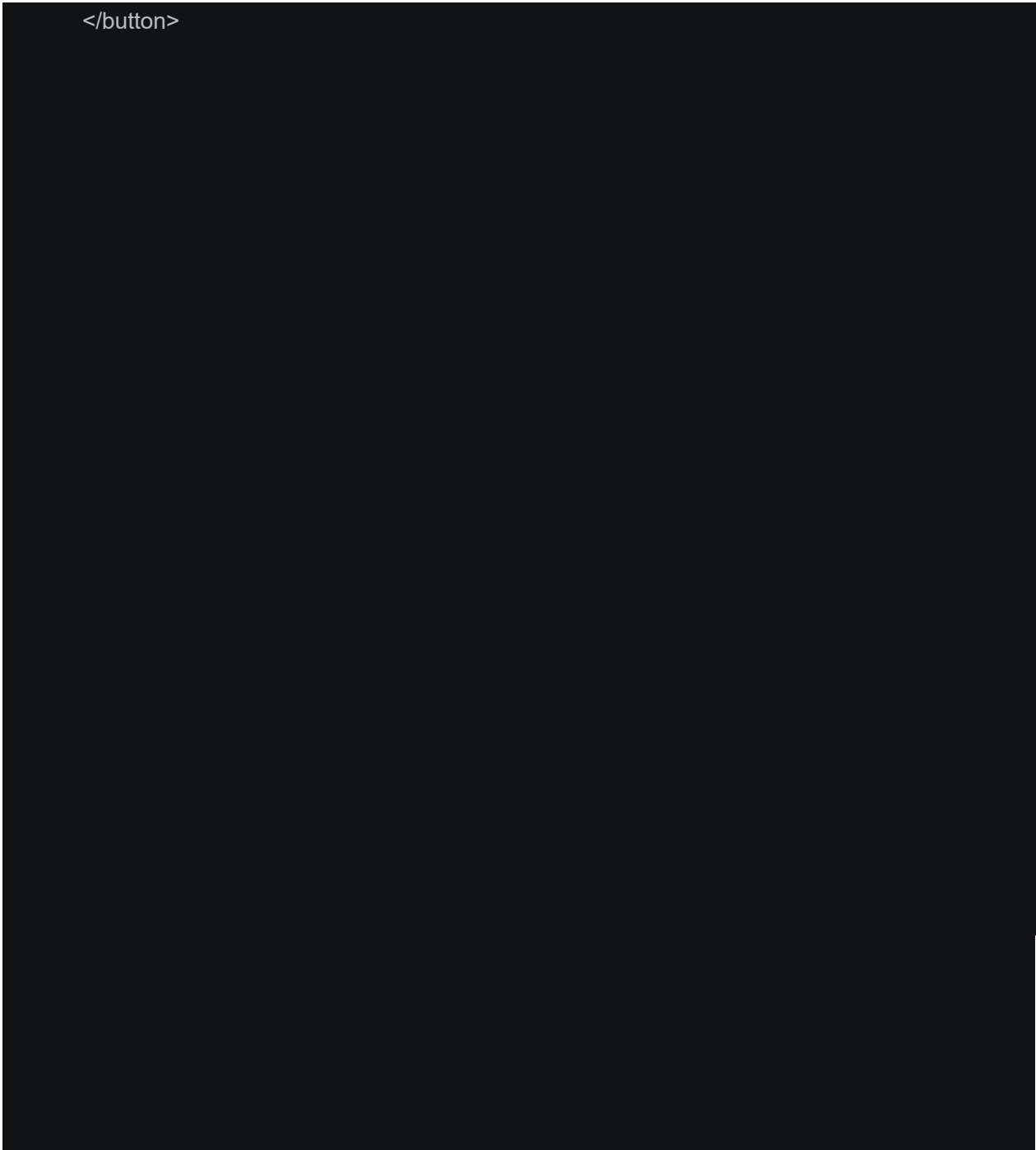
<p id="description">
  I enjoy creating useful applications
and exploring new technologies.
</p>

<div class="buttons">

  <button onclick="changeName()">
    Change Name
```



</button>





```
<button onclick="changeRole()">
```

Change Role

```
</button>
```

```
<button onclick="changeDescription()">
```

Change Bio

```
</button>
```

```
<button onclick="changeAvatar()">
```

Change Avatar

```
</button>
```





```
<button onclick="changeTheme()">
  Change Theme
</button>

</div>

<div class="status" id="status">
  DOM is ready. Try the buttons!
</div>

</div>

<script>

  // Change content

  function changeName() {

    const name =
document.getElementById("name");

    name.innerText =
"Almas Mohammed";
```

```
document.getElementById("status").innerText =  
    "Name changed using the DOM.";
```

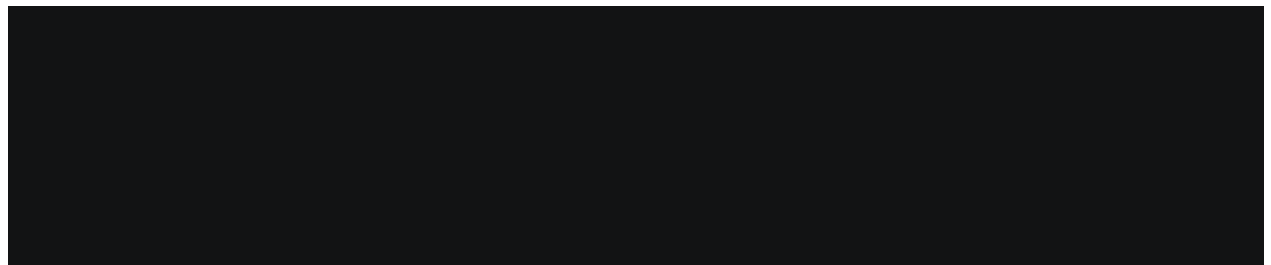


```
}
```

```
// Change content and style
```

```
function changeRole() {  
  const role =  
document.getElementById("role");
```

```
  role.innerText =  
"Creative AI Developer";
```



```
role.style.color
=
    "#7b4f35";

    document.getElementById("status").innerText =
"Role and style changed dynamically.";
    }

    // Change content and style

    function changeDescription() {

        const description =
document.getElementById("description");

        description.innerText =
            "I love creating useful applications and exploring AI.";

        description.style.fontSize =
            "17px";

        document.getElementById("status").innerText =
"Bio and font size changed.";
    }
```

[REDACTED]

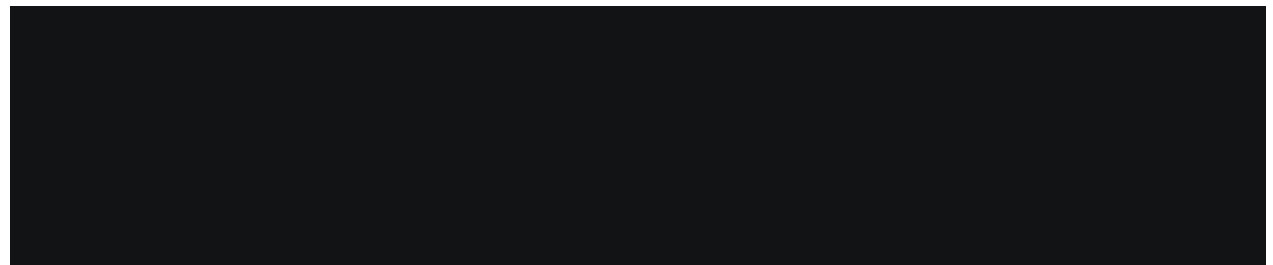
[REDACTED]



```
// Change avatar style

function changeAvatar() {

    const avatar =
document.getElementById("avatar");
```





```
avatar.style.background =  
  "#d9b6e8";
```

```
avatar.style.borderColor =  
  "#9259a8";
```

```

        avatar.style.transform =
"rotate(3deg) scale(1.05)";

        avatar.setAttribute(
            "title",
            "Updated pixel avatar"
        );

        document.getElementById("status").innerText =
"Avatar style and attribute changed.";
    }

    // Change page theme

    function changeTheme() {

        document.body.classList.toggle("dark");

        document.getElementById("status").innerText =
            "Theme changed using the DOM.";
    }

</script>

</body>

</html>

```

7. CODE EXPLANATION

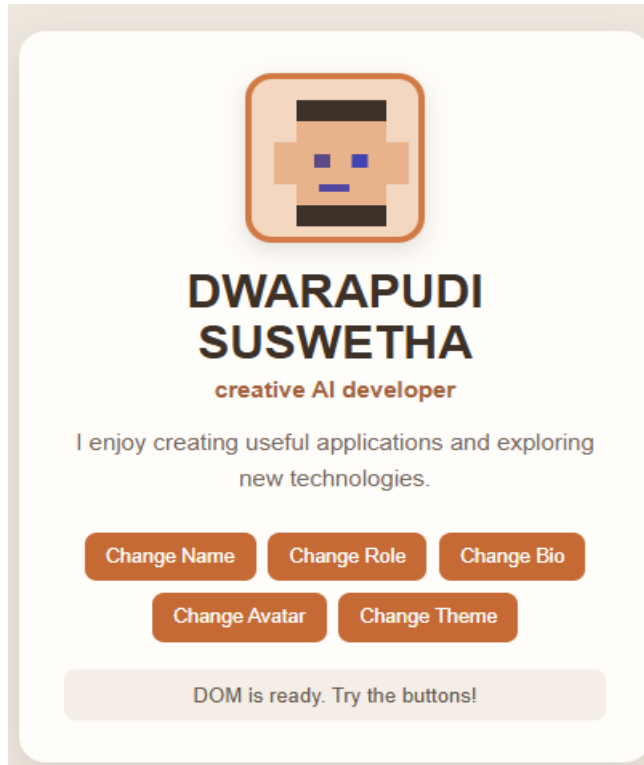
Code / Function	Explanation
<code>document.getElementById()</code>	Selects an HTML element using its unique ID.

<code>innerText</code>	Changes the text content of an HTML element.
<code>style.color</code>	Dynamically changes the text color.
<code>style.fontSize</code>	Dynamically changes the font size.
<code>style.background</code>	Changes the background color of an element.
<code>style.borderColor</code>	Changes the border color of the avatar.
<code>style.transform</code>	Applies rotation and scaling to the avatar.
<code>setAttribute()</code>	Changes or sets an HTML attribute.
<code>classList.toggle()</code>	Adds or removes a CSS class dynamically.
<code>changeName()</code>	Changes the profile name using DOM manipulation.
<code>changeRole()</code>	Changes the role text and its color.
<code>changeDescription()</code>	Changes the profile description and font size.
<code>changeAvatar()</code>	Changes avatar styling and its title attribute.
<code>changeTheme()</code>	Switches between light and dark themes.
<code>onclick</code>	Calls a JavaScript function when a button is clicked.

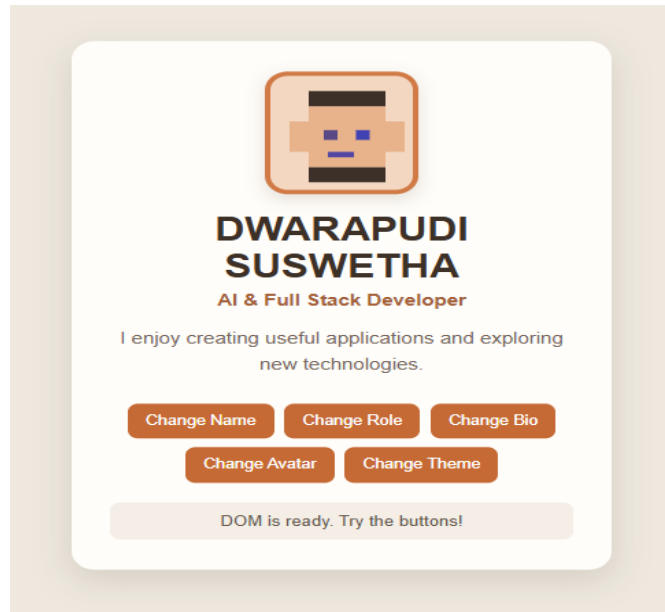
8. TEST CASES AND EXECUTION DATA

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Click Change Name	Name should change to "Almas Mohammed"	Name changed successfully	Pass
TC-02	Click Change Role	Role should change to "Creative AI Developer"	Role changed successfully	Pass
TC-03	Click Change Avatar	Avatar background and border should change	Avatar style changed	Pass
TC-04	Click Change Theme	Page should switch to dark theme	Dark theme applied	Pass

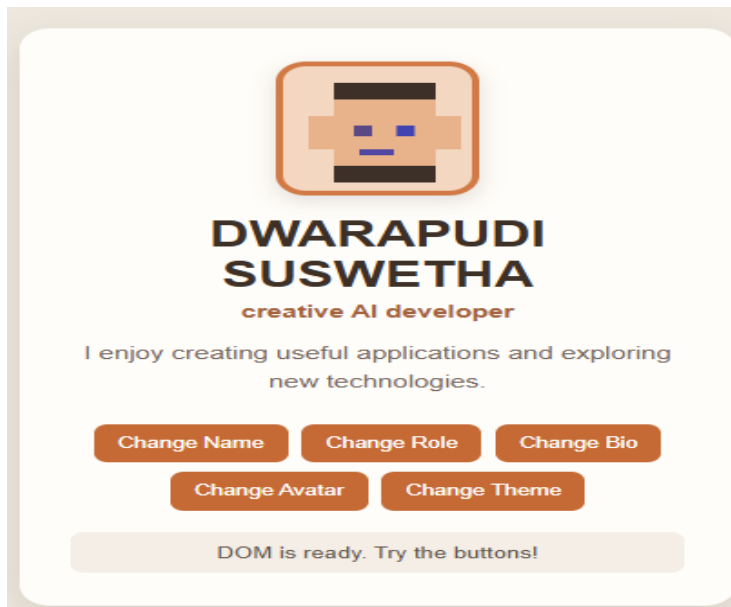
TC-01 output:



TC-02 Output:



9. OUTPUT:



10. RESULT

Thus, an interactive **DOM Profile Customizer** was successfully developed using **HTML, CSS and JavaScript**. The application successfully demonstrated dynamic modification of HTML content, CSS styles, HTML attributes and CSS classes using JavaScript DOM manipulation.
